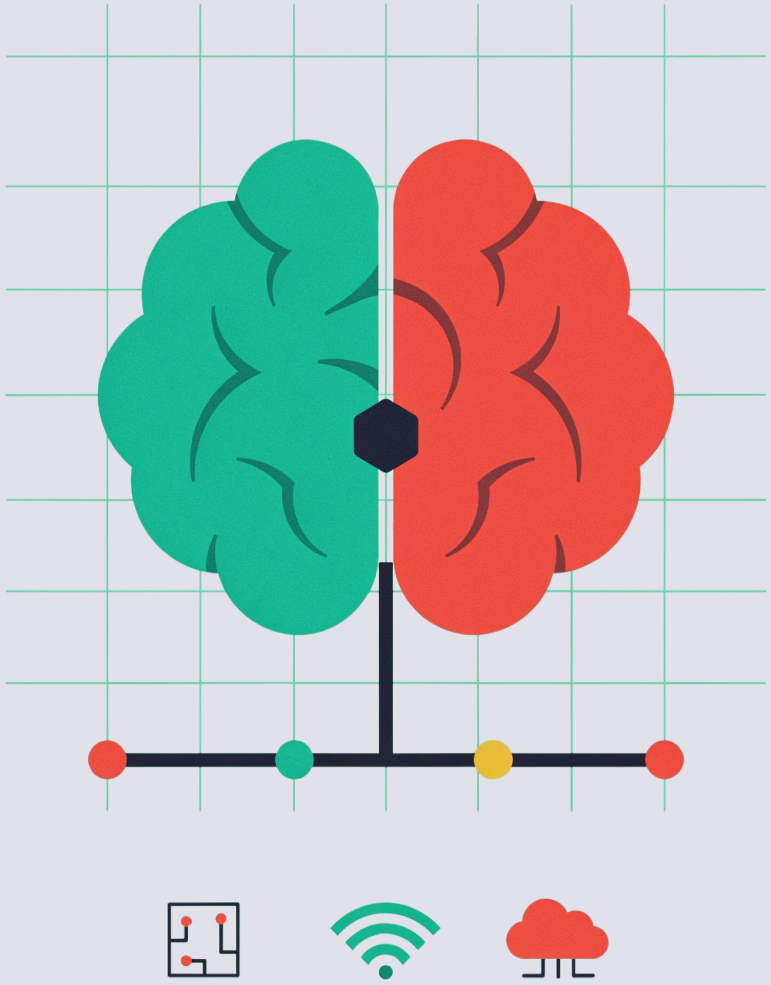


Building an AI Agent Using MCP Servers

Goal

Convert plain English instructions into automated business workflows

Automation spans

Browser actions, database queries, API calls, and file operations

Achieved by

Combining LLMs with MCP servers

Example Automation Workflow

User gives a plain English prompt describing the task

Agent handles it end-to-end, with no code required:

01

Query database for registration data

02

Fill registration form in browser

03

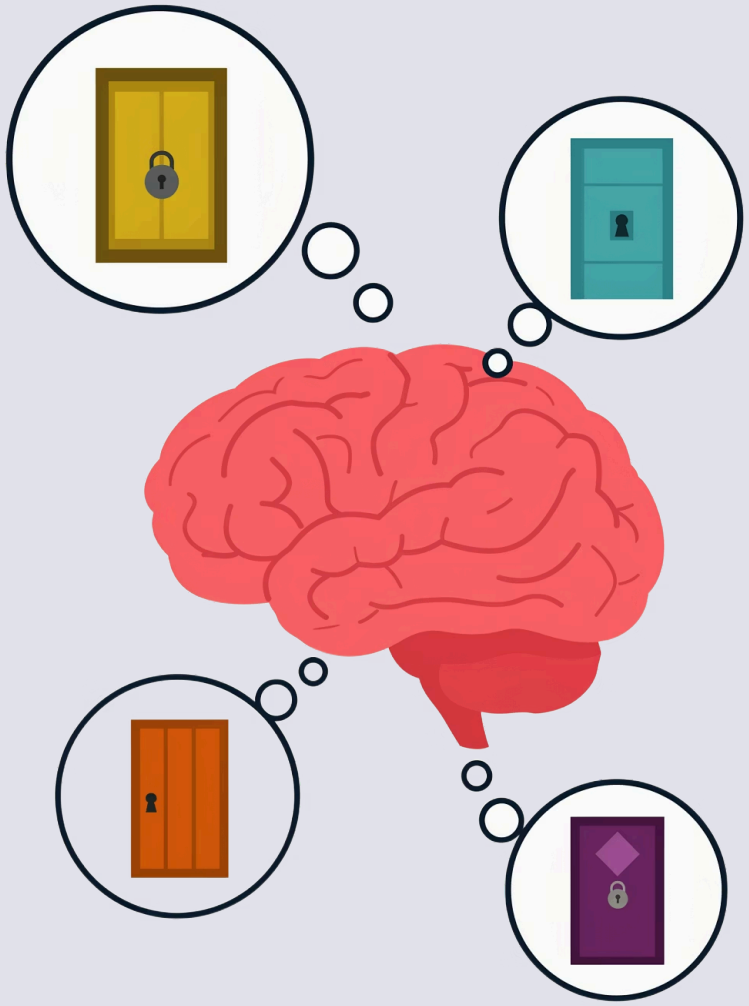
Call REST API to verify registration

04

Store results in Excel file

05

Complete login automation



The Role — and Limits — of LLMs

LLMs process prompts, understand intent, and reason about next steps

But they cannot directly access systems:

What LLMs Can Do

- Process prompts
- Understand intent
- Reason about next steps

What LLMs Cannot Do

- Can't query databases
- Can't automate browsers
- Can't call APIs
- Can't access local files

They can only reason — not act

Why MCP Is Needed

Connects LLMs with external systems and tools

MCP (Model Context Protocol) connects LLMs with external systems and tools

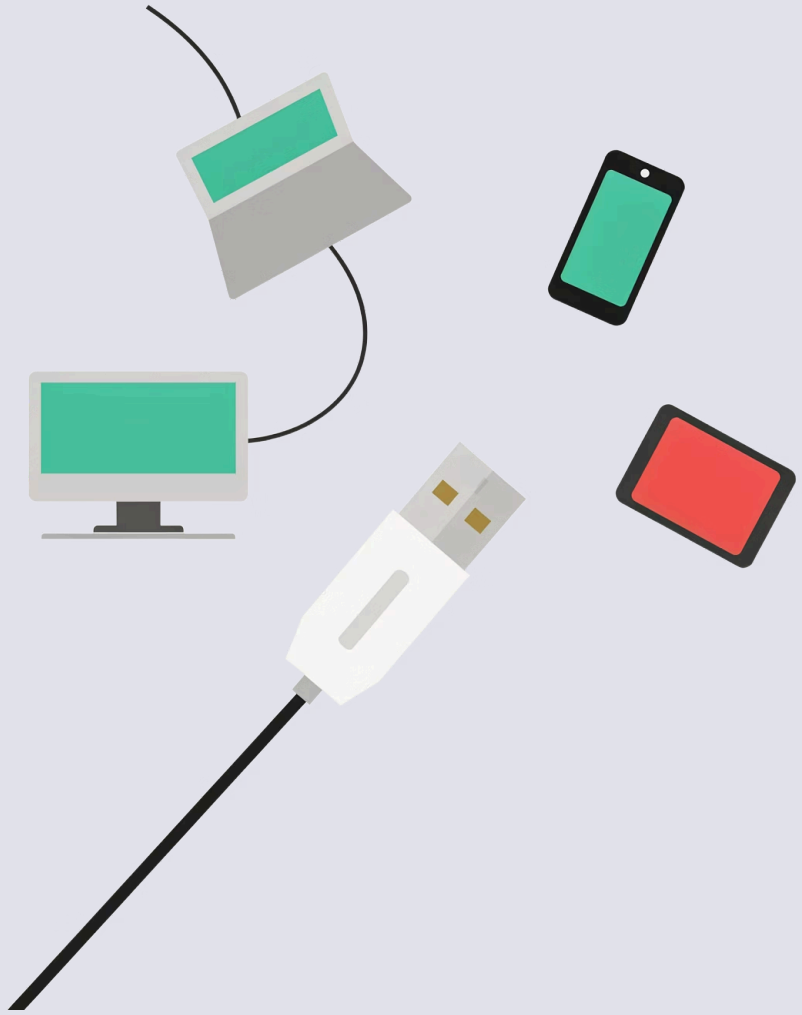
Provides context LLMs otherwise lack

Provides context LLMs otherwise lack

Enables interaction with real-world resources

Enables interaction with real-world resources





What Is MCP?

Open Protocol

An open protocol standardizing how applications provide context to LLMs

AI Interaction

Lets AI systems interact with external tools and data sources

Universal Connector

Works like a universal connector — the "USB-C" for AI systems

The Context Problem Without MCP

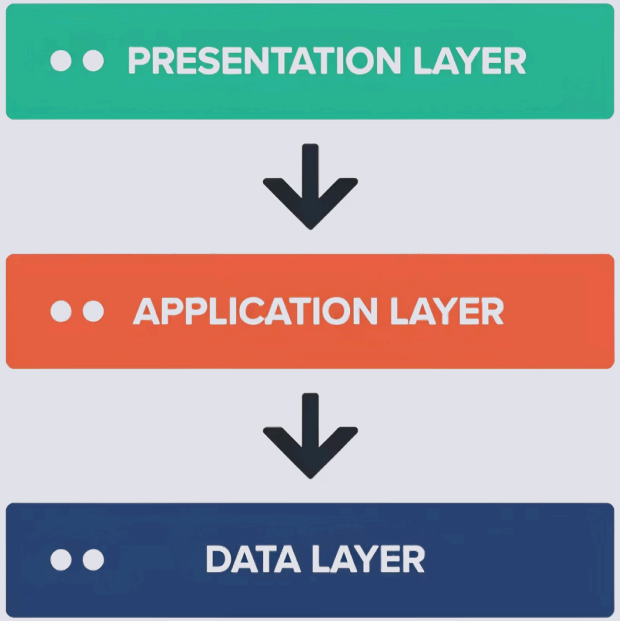
LLMs can't answer questions about private data

> *"How many total orders are in my database?"*

Reason: no access to internal systems

MCP bridges this gap by connecting LLMs to real resources





MCP Architecture

1

MCP Client

The interface users interact with (Claude, ChatGPT, Gemini)

2

MCP Server

The middle layer connecting LLMs to external systems

3

External Resources

Databases, browsers, file systems, APIs

How MCP Works

01

User asks the LLM a question

02

LLM interprets intent using natural language understanding

03

LLM communicates with the MCP server via a standardized protocol

04

MCP server calls the relevant tool functions

05

Tool functions interact with external systems

06

Results return to the LLM and are presented to the user



Tool Functions in MCP Servers

MCP servers expose tools (functions) LLMs can call

Each tool performs one specific operation:

Execute SQL query

Navigate browser / click elements

Read files

Call APIs

LLM selects the right tool based on its description

Example — Database Query with MCP

01

LLM receives a question about customer orders

02

Checks available MCP tools → finds execute_sql

03

Inspects database structure (tables, columns, relationships)

04

Generates the appropriate SQL query

05

MCP executes it and returns results

Example — Browser Automation with MCP

Browser automation MCP server exposes functions:

browser_navigate	**browser_click**	**browser_type**	**browser_go_back**

LLM selects functions based on the instruction

> *"Go to a website and run a smoke test"*

MCP executes the browser automation steps

Connecting MCP Servers

Configured via a simple JSON file — just a few lines

Once set up, the LLM auto-detects available tools

Multiple MCP servers can run together:

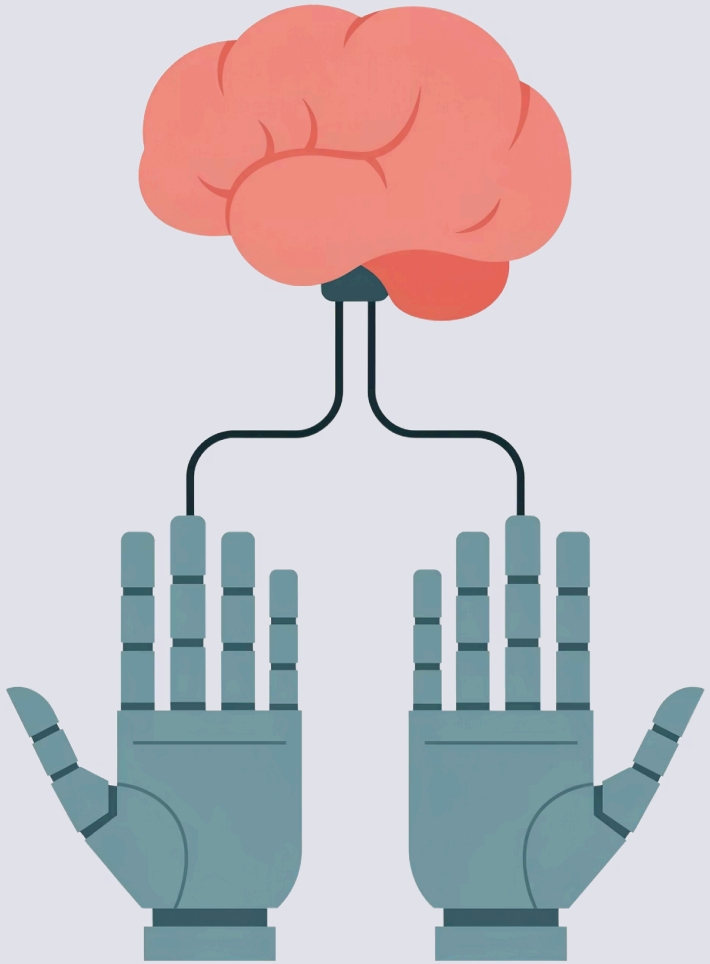
Database

Browser automation

File system

API testing

Excel manipulation



AI Agent = Brain + Hands

LLM

Reasoning engine (the brain): decides what to do

MCP tools

Execution capabilities (the hands): perform the actual actions

Together: plain English instructions trigger complex, multi-system workflows automatically

Practical Example — Full Workflow

01

Retrieve user data from database

02

Fill web registration form

03

Verify success through API

04

Store results in Excel

05

Run browser automation tests